Zcim Project

Design Specifications

draft 2025-08-01

Introduction

What is Zcim?

Zcim is a backronym that stands for **Z**80 **C**P/M **I**mitation **M**achine, pronounced “Zee Sim.” The goal is to create an imitation of a Z80 CP/M computer system that is in a more accessible wrapper than a traditional system emulator.

The bulk of the system is in AssemblyScript. The AssemblyScript code is compiled into WebAssembly (WASM). The top-level control is written in TypeScript. Since the WebAssembly code is executed inside a sandbox, the TypeScript code is the only part that actually performs I/O.

References

The listed references provide additional information related to Zcim software.

Z80/8080/8085 Central Processing Unit (CPU)

These documents are related to hardware operation of the CPU.

- Decoding Z80 Opcodes by Cristian Dinu.
- Zilog Z80 CPU User Manual UM008011-0816
- Z80 Undocumented Features. Version 0.3. by Sean Young
- Z80 MEMPTR
- 8080 Microcomputer Systems User’s Manual
- 8080/8085 Assembly Language Programming Manual

CP/M

Source code and other documents about the CP/M operating system.

- CP/M 1.4 Interface Guide
- CP/M 1.4 System Alteration Guide
- CP/M 2.2 Manual
- CP/M Version 3 Operating System - Programmers Guide
- CP/M Version 3.0 Operating System - System Guide

Prior Art

Sites about existing emulators.

- AltairZ80
- ares
- open-simh
- yaze
- yaze-ag
- z80pack

Testing
Documents and code related to testing.

- jsmoo
- ProcessorTests

ZCim Project
Other Zcim Project documents which may be of interest.

- BDOS Specifications

Scope
The purpose of this document is to explain how the Zcim software is expected to function, and how it was built to achieve its aims. The audience of the document is primarily people who are involved in the implementation, or in the evaluation of the software. It is not expected that a typical user of Zcim would be interested in this level of detail.

As a design document, the text assumes some knowledge of software development. It is not meant as a tutorial about software, although it should serve as a tutorial on the Zcim software itself.

Basic Design
There are interacting state machines that implement Zcim.

- Top level control
- Virtual CP/M layer
- Virtual Z80
- Virtual Devices

Top-level Controls
There are two top-level control implementations. They are both written in TypeScript. They interface to the AssemblyScript code using WebAssembly (WASM).

The two versions represent different user interfaces and environments. There is a node version that is executed from the command-line and uses command-line options, environment variables and the like to determine what the configuration and exact operation requested. The console of Zcim is handled through the standard input and standard output files.

The second version is designed to be embedded into a web page. The console of Zcim is handled through a terminal emulation window The general operation is the same for both top-level controls. The control must configure the virtual CP/M system and its underlying emulated hardware, and also handle any I/O.

Since the lower layers live inside the WASM sandbox, the top-level is the only part that can perform I/O of any kind. In many ways, the structure of the system revolves around this.

How the control performs the configuration depends upon the version. The node version uses command-line options, remote configurations as well as hard coded default values. It is normally used to execute a CP/M command using a local file system as an emulated disk.

`zcim` command-line interface

The `zcim` command-line interface is a `node` program, although a `deno` or `bun` version should be possible.

The `zcim` command includes support for a number of functions which include acting as a virtual CP/M environment. The `run` command will combine command-line arguments with default settings, and optionally settings downloaded from shared resources to construct a `SystemConfig`.

The program's `standard input` becomes the console input (`CONIN`) for the Virtual CP/M layer, with the `standard output` becoming the console output (`CONOUT`). The Virtual CP/M aux/reader input, aux/punch output, and list/printer output are normally routed to `/dev/null`, but this can be overridden using the standard `settings` of the `SystemConfig`.

Commands

- run file args
- fs check
- fs copy
- fs dir
- fs dump
- fs get
- fs getattr
- fs put
- fs setattr
- fs stat
- fs users
- disk putsys
- disk getsys
- disk dump
- disk convert
- disk check
- ar dir
- ar copy
- ar json
- ar check
- ar convert

web-based interface
The web component may fetch remote configurations with overrides set via component attributes. It is normally used to execute one or more CP/M commands using an emulated disk that is initialized from the contents of a file.

Virtual CP/M layer

An AssemblyScript layer that imitates the Digital Research CP/M Operating System. There is no actual CP/M code executed by the Zcim system.

The system has the concept of an **OS flavor**. The initial CP/M flavor supported is **CP/M 2.2**. The **CP/M 1.4** and **CP/M 3.0** flavor support is planned.

The emulated Z80 system (`Z80CPU`) supports both I/O traps as well as breakpoints. The Z80 layer is setup with breakpoints set for Basic Disk OS (BDOS) and Basic I/O System (BIOS) jump tables. This allows the typical application to execute in mode much like a supervisor/user operating system. The emulation executes the “user” code until a call to the emulated **BIOS** jump table or the **BDOS** entry point.

This will then trigger a breakpoint. As part of the breakpoint handler, the **Virtual CP/M** layer executes code that performs the expected operation and updates the emulated system. Then the layer simply **returns** from the breakpoint with the results already in place.

This means that from the CP/M application program point of view, the CP/M BDOS is a `CALL` from the application to location `0x0005`, which contains a `JMP` (Jump), which targets another `JMP` (which triggers an emulation breakpoint) that then targets a `RET` instruction at the highest location in memory (typically `0xFFFF`).

The Virtual CP/M layer supports two forms of virtual disks:

- Local Directory
- Disk Image

Local Directory Virtual Disks
A local directory may be configured as a virtual CP/M disk.

Note Direct BIOS I/O to a local directory disk is **not** currently supported.

The filenames from the existing local files will be mapped into their CP/M equivalents. Any I/O to CP/M files will actually use these existing files.

Warning The CP/M to local directory file mapping looses information.

It does not support:

- high order bit flags (F1, F2, F3, F4, T1, T2, and T3)
- exact file sizes
- non-ASCII characters

Other limitations include:

If there are too many files, or too many name collisions, some local files may not be accessible to the Virtual CP/M.

Local files that are too large for CP/M are not accessible from Virtual CP/M.

If there are too many files in the directory, Virtual CP/M will not be able to create an additional files.

The mapping is predictable in the absence of collisions, but once collisions occur, it is unlikely that someone would be able to predict which file is which, and the mapping would be redone on each load.

Virtual Disk Images

A traditional disk image file may be used to imitate a floppy or hard disk. Disk images may optionally be loaded from shared resources via URL, or local files. It is also possible to configure a **scratch disk**: A virtual disk that is initialized as empty and is not persisted.

Virtual Z80 (CPU) layer

An AssemblyScript layer that imitates a Z80 CPU, a virtual bus and a collection of one or more virtual devices. The initial system emulates three different CPU implementations (all Z80′s): **NMOS**, **CMOS** and **BM1**. Support of **8080A** and **8085** CPU emulation is planned.

Virtual Device layer

In general, the virtual devices are limited in scope. The goal of Zcim is imitation rather than emulation. While the interfaces are designed to allow for a more detailed emulation of devices, the initial implementation has a modest scope. So the devices are the minimum needed for a virtual CP/M system.

A more traditional system emulator may be used if the exact replication of historical hardware is needed.

Memory Devices

- Standard RAM
- ROM
- Banked RAM (planned)

I/O Devices

- Processor Test I/O Device
- Virtual UART (planned)

Memory Map

The initial implementation supported a whopping 64K of RAM. Banked RAM is planned with the CP/M 3.0 Flavor feature release.

Low Memory Layout

The low memory layout matches a typical CP/M 2.2 system. None of the memory areas reserved for BIOS use are changed. In a typical system, none of the restart vectors are used for interrupts, since the system does not use them for I/O.

BDOS and BIOS Entry Points

The actual work of the imitated CP/M operating system is not performed using emulated Z80 instructions. Instead, the BDOS and BIOS entry points in high memory are simply a jump instruction (`JMP`) to a return instruction (`RET`). The virtual Z80 CPU is configured with a breakpoint at the entry. The breakpoint handler will then execute the operating system call, and when the operation is complete, it will resume operation, execute the jump instruction, followed by returning to the user code.

BIOS Data Area

Resident Data

CP/M 1.4 Flavor Specific

CP/M 2.2 Flavor Specific

CP/M 3.0 Flavor Specific

System Control Block Data

The System Control Block (SCB) was introduced in CP/M 3.0. It contains the shared BDOS state that CP/M maintains. While the SCB was only used in CP/M 3.0, it is used internally in the virtual CP/M layer for all flavors.

Offset	Size	Id	Equate	Description
0x00	Byte	`HashLength`	`hashl`	hash length (0,2,3)
0x01	2 * Word	`Hash0` / `Hash1`	`hash`	hash entry
0x05	Byte	`BDOSVersion`	`bdos$version`	BDOS Version number
0x06	2 * Word	-	`util$flgs`	utility flags (reserved)
0x0A	2 * Word	-	`dspl$flgs`	display flags (reserved)
0x0E	Byte	-	`clp$flgs`	CLP flags
0x0F	Byte	`SubmitFileDrive`	`clp$drv`	Submit file drive number
0x10	Word	`ProgramReturnCode`	`prog$ret$code`	program return code
0x12	Byte	`MultipleCommandBufferPage`	`multi$rsx$pg`	multiple command buffer page
0x13	Byte	`CCPDrive`	`ccpdrv`	ccp default drive
0x14	Byte	`CCPUser`	`ccpusr`	ccp default user number
0x15	Word	`CCPBufferAddress`	`ccpconbuf`	ccp console buffer address

Offset	Size	Id	Equate	Description
0x17	Byte	CCPFlag1	ccpflag1	ccp flags byte 1
0x18	Byte	CCPFlag2	ccpflag2	ccp flags byte 2
0x19	Byte	CCPFlag3	ccpflag3	ccp flags byte 3
0x1A	Byte	ConsoleWidth	conwidth	console width
0x1B	Byte	ConsoleColumn	concolumn	console column position
0x1C	Byte	ConsolePageLength	conpage	console page length (lines)
0x1D	Byte	ConsoleLine	conline	current console line number
0x1E	Word	ConsoleInputBufferAddress	conbuffer	console input buffer address
0x20	Word	ConsoleInputBufferLength	conbuffl	console input buffer length
0x22	Word	ConsoleInRedirection	conin$rflg	console input redirection flag
0x24	Word	ConsoleOutRedirection	conout$rflg	console output redirection flag
0x26	Word	AuxInRedirection	auxin$rflg	auxillary input redirection flag
0x28	Word	AuxOutRedirection	auxout$rflg	auxillary output redirection flag
0x2A	Word	ListOutRedirection	listout$rflg	list output redirection flag
0x2C	Byte	PageMode	page$mode	page mode flag 0=on, 0ffH=off
0x2D	Byte	PageModeDefault	page$def	page mode default
0x2E	Byte	BackspaceFlag	ctlh$act	ctl-h active
0x2F	Byte	DeleteFlag	rubout$act	rubout active (boolean)
0x30	Byte	TypeAheadFlag	type$ahead	type ahead active
0x31	Word	-	contran	console translation subroutine
0x33	Word	ConsoleMode	con$mode	console mode (raw/ cooked)
0x35	Word	BDOSBuffer	ten$buffer	128 byte buffer available to banked BIOS
0x37	Byte	OutputDelimiter	outdelim	output delimiter
0x38	Byte	ListOutputFlag	listcp	list output flag (ctl-p)

Offset	Size	Id	Equate	Description
0x39	Byte	ScrollFlag	q$flag	queue flag for type ahead
0x3A	Word	SCBAddress	scbad	system control block address
0x3C	Word	DMAAddress	dmaad	dma address
0x3E	Byte	Disk	seldsk	current disk
0x3F	Word	BDOSInfo	info	BDOS variable “info”
0x41	Byte	FCBErrorFlag	resel	disk reselect flag
0x42	Byte	SameDiskFlag	relog	relog flag
0x43	Byte	BDOSFunction	fx	function number
0x44	Byte	UserNumber	usrcode	current user number
0x45	Word	NextDirectory	dcnt	directory record number
0x47	Word	SearchFCB	searcha	fcb address for searchn function
0x49	Byte	SearchType	searchl	scan length for search functions
0x4A	Byte	MultiSectorCount	multcnt	multi-sector I/O count
0x4B	Byte	BDOSErrorMode	errormode	BDOS error mode
0x4C	Byte	DriveSearch0	drv0	search chain - 1st drive
0x4D	Byte	DriveSearch1	drv1	search chain - 2nd drive
0x4E	Byte	DriveSearch2	drv2	search chain - 3rd drive
0x4F	Byte	DriveSearch3	drv3	search chain - 4th drive
0x50	Byte	TemporaryFileDrive	tempdrv	temporary file drive
0x51	Byte	ErrorDrive	errdrv	error drive
0x52	Word	-	-	Unknown
0x54	Byte	OpenDoorFlag	media$flag	drive door open flag
0x55	Word	-	-	Unknown
0x57	Byte	BDOSFlags	bdos$flags	BDOS flags
0x58	Word	DayNumber	date	date stamp
0x5A	Byte	HoursBCD	-	Hour (BCD)
0x5B	Byte	MinutesBCD	-	Minutes (BCD)
0x5C	Byte	SecondsBCD	-	Seconds (BCD)
0x5D	Word	CommonMemoryBase	com$base	common memory base address
0x5F	Byte	JMP	error	JMP opcode
0x60	Word	BDOSErrorRoutine	error$jmp	BDOS error routine address

Offset	Size	Id	Equate	Description
0x62	Word	BDOSAddress	top$tpa	top of user TPA (address at 6,7)